# NGO Volunteer Connect Hub - System Documentation

This document provides a detailed report on the programming languages, technologies, database schemas, and system requirements used in the NGO Volunteer Connect Hub platform.

## 1. Programming Languages & Technologies

The platform is built using a modern full-stack web architecture, using the following languages and technologies:

### 1.1. TypeScript (TSX / TS)

- Purpose: TypeScript is the core language of the application, used to write type-safe, maintainable code for both client and server logic.
- Frontend (TSX): React components, TanStack Router route definitions (e.g., in src/routes/), and layout templates.
- Server Functions (TS): Server-side logic (e.g., in src/lib/certificate.functions.ts and src/lib/geocode.functions.ts) built with createServerFn to securely perform actions (PDF generation, database updates, external geocoding calls) that require admin credentials or server execution.
- Configuration (TS): Build settings defined in vite.config.ts.

### 1.2. HTML

- Purpose: Standard markup language.
- Usage: Entrypoint template to bootstrap the React application mount point.

### 1.3. CSS (Tailwind CSS v4)

- Purpose: Styling and responsive design.
- Usage: Tailored stylesheet (src/styles.css) utilizing Tailwind v4 features to ensure highly performant, custom-themed layouts, dynamic hover animations, and cohesive grid/flex structures.

### 1.4. SQL (PostgreSQL Dialect)

- Purpose: Database schema creation, data manipulation, Row Level Security (RLS) access rules, triggers, and server-side RPC functions.
- Usage: Migration scripts under supabase/migrations/ that configure database schemas, set up constraints, define security policies, and implement serverless business logic (such as auto-updating volunteer statistics upon attendance updates).

### 1.5. Configuration Formats (JSON / TOML)

- Purpose: Project metadata and compiler settings.
- package.json / package-lock.json: Manage npm dependencies, scripts, and runtime engines.
- tsconfig.json: Configures TypeScript compiler rules (strict types, path aliases).

- netlify.toml: Hard-pins Netlify build targets to ensure npm builds deploy correctly.

## 2. Database Schema (Supabase / PostgreSQL)

The backend database runs on Supabase (PostgreSQL) and uses custom Enums, Tables, Row Level Security (RLS) policies, Triggers, and Database RPC Functions.

### 2.1. Enumerated Types (Enums)

- public.app_role: Defines user account types: 'admin', 'ngo', 'volunteer'.
- public.verification_status: NGO verification states: 'pending', 'verified', 'rejected'.
- public.event_status: Tracks event state: 'published', 'cancelled', 'completed'.
- public.application_status: Tracks volunteer applications: 'pending', 'accepted', 'rejected', 'waitlisted', 'cancelled'.

### 2.2. Database Tables

- public.profiles: Extends standard user authentication info (maps to auth.users). Stores id, full_name, phone, pincode, email, coordinates (lat, lng), and avatar_url.
- public.user_roles: Maps users to their roles.
- public.volunteer_profiles: Tracks volunteer-specific data, including skills (text array), availability (JSONB), bio, events_completed, and hours_total.
- public.ngo_profiles: Stores NGO details, including org_name, registration_number, registration_cert_url, contact_email, contact_phone, address, coordinates (lat, lng), and verification_status.
- public.events: Store event listings created by NGOs. Attributes include title, description, category, banner_url, starts_at, ends_at, address, coordinates (lat, lng), total_spots, contact details, and status.
- public.event_roles: Specific volunteer roles within an event, defining role_name, spots_needed, and spots_filled.
- public.applications: Link volunteers to events and roles. Includes application message, status, and timestamps.
- public.attendance: Connects to applications to log if the volunteer attended and the number of hours contributed.
- public.certificates: Stores URLs of generated PDF certificates, linking them to volunteer applications.
- public.notifications: Stores user-facing alerts, with type, title, body, link redirect destination, and read_at status.
- public.admin_reports: Allows reporting of events or NGOs, containing reported_entity_type, reason, status, and reporter_id.

### 2.3. Automated Database Triggers

- on_auth_user_created: Executes handle_new_user() on sign-up to automatically insert profile entries and assign user roles.
- trg_application_change: Executes handle_application_change() on application updates. Increments or decrements spots_filled on the associated event_roles, sends notification updates, and automatically promotes waitlisted applicants if a spot becomes available.
- trg_new_application: Executes handle_new_application() to notify NGOs when a volunteer applies to their events.

- trg_attendance_change: Executes handle_attendance_change() to recalculate a volunteer's events_completed and hours_total stats whenever attendance records are inserted or edited.

### 2.4. Custom RPC Functions

- haversine_km: Computes geographical distance between coordinates.
- search_events: Advanced query endpoint. Searches for events within a specific radius of a user's location, filtered by category, time bounds, and spot availability.

## 3. System Requirements & Architecture

The application requires specific runtime configurations to support SSR (Server-Side Rendering) and serverless database requests:

### 3.1. Frontend Requirements

- React 19: Modern React framework for components and hooks.
- TanStack Router & Start: Type-safe client-side routing, page layout shells, SSR engine, and server function executors.
- TanStack Query (React Query): Client-side data fetching, automatic caching, caching invalidation, and data synchronization.

### 3.2. Backend & Deployment Requirements

- Node.js: The project requires Node.js (version 18 or above) to run build and compilation commands.
- Supabase Client SDK: Communicates with the Supabase PostgreSQL database, handles user authentication sessions, and handles file uploads to storage buckets.
- Netlify Platform: Hosting target. Nitro compiles the SSR server bundles and automatically registers them as Netlify serverless functions, serving static assets from the dist directory.